PHASE

2

Getting to C: Screen, Serial and kprintf

Give the kernel a stack, jump into C, and build the two output channels used for debugging for the rest of the project

Depends on	Phase 1: a Multiboot kernel that boots, a Makefile, GDB workflow
Delivers	A kernel written mostly in C that prints a coloured banner and the machine's memory size on the VGA text screen, mirrors every message to the serial port (so it appears in the host terminal), and has `kprintf`, `panic` and `ASSERT`.

1 Scope and motivation

From now on, most of Kagan is written in C. Getting there takes one small piece of assembly (a stack) and a few compiler flags. Right after that comes the most important debugging tool in the kernel: a way to print. With no operating system underneath, `printf` does not exist, so this phase writes the drivers it would normally sit on.

The phase builds two output channels. The **VGA text screen** is what a user sees. The **serial port** is what a developer needs: QEMU can redirect it to a terminal or a log file, where messages can be scrolled back, searched, and kept after a crash. When something goes wrong in Phase 6 or Phase 8, the serial log is usually the only witness. Both drivers also introduce the two ways a CPU talks to devices, memory-mapped I/O and port I/O, which every later driver uses.

2 Background

2.1 What C needs before `kmain` can run

A C function expects three things from its environment. It expects a **valid stack**, because every call pushes a return address and locals live there; Multiboot leaves ESP undefined, so the loader must reserve memory for a stack and point ESP at its top. It expects **zeroed `.bss`**, because the C standard says globals without an initialiser start at zero; GRUB and QEMU load ELF kernels by their program headers and zero-fill the part of each segment beyond the file contents, so `.bss` is handled by the loader, but it is worth confirming once in GDB. And it expects **the calling convention**: arguments on the stack, so the loader can pass the Multiboot magic and information pointer to `kmain` by pushing them.

2.2 Two ways to talk to a device

Memory-mapped I/O places a device's memory or registers at ordinary physical addresses. The VGA text buffer lives at `0xB8000`: write a byte there and a character appears on screen. Access uses normal pointer operations, marked `volatile` so the compiler cannot optimise them away. **Port-mapped I/O** uses a separate 64 K address space reachable only through the `in` and `out` instructions. The VGA cursor and the serial port are controlled this way. MOS §5.1.3 compares the two designs and explains why x86 has both.

2.3 The VGA text buffer

In text mode the screen is 80 columns by 25 rows. Each cell is two bytes: the character (in code page 437, which extends ASCII) and an attribute byte whose low four bits are the foreground colour and whose high four bits are the background (Figure 1). Cell (row, col) is at index $row \times 80 + col$. The blinking hardware cursor is separate: it is positioned by writing a 16-bit cell index to the CRT

controller through ports `0x3D4` (register select) and `0x3D5` (data), register 14 for the high byte and 15 for the low byte.

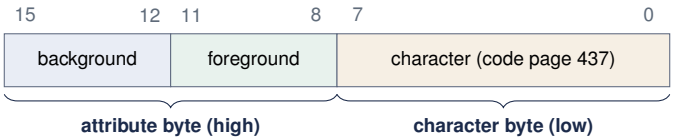


Figure 1: One 16-bit VGA text cell. White on blue is attribute `0x1F`; the cell for “A” is then `0x1F41`.

2.4 The serial port

A PC’s first serial port (COM1) is a 16550-compatible UART with its registers at I/O ports `0x3F8`–`0x3FF`. It is initialised once, choosing the speed with a divisor of its 115 200 Hz base clock and the frame format (8 data bits, no parity, one stop bit, known as “8N1”). After that, sending a byte means waiting until the *line status* register says the transmitter is empty, then writing the byte to the data port. QEMU’s `-serial stdio` connects this port to the host terminal.

2.5 A console abstraction and kprintf

Rather than making every function choose where to print, a single `kprintf` that formats a string and hands each character to a small console layer, which forwards it to every registered output (Figure 2). Formatting uses `stdarg.h`, which is available in freestanding mode because it is implemented by the compiler itself.

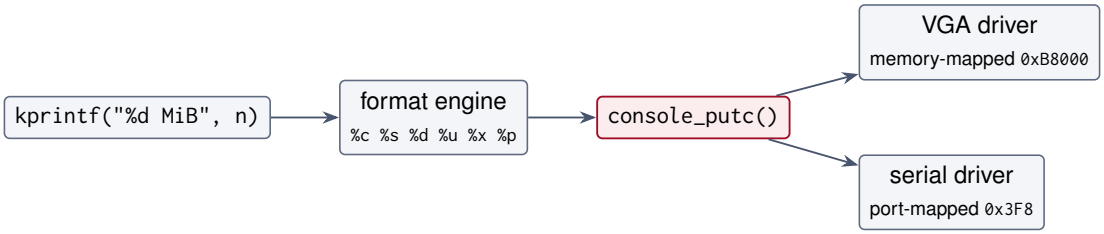


Figure 2: Output path. Adding a third sink later (a log buffer for `dmesg`) is one line.

3 References

Weight	Source	Section	Pages	Relevance
PRIMARY	LOB	Ch. 3 *Getting to C*: stack, calling C from assembly, packing structs, compiling C, build tools	19–22	Tasks 2.1–2.3 almost step by step.
PRIMARY	LOB	Ch. 4 *Output*: framebuffer, cursor, serial port, configuring the line, buffers and modem	23–31	The two drivers of this phase.
PRIMARY	OSTEP	Ch. 36 *I/O Devices* (§36.1–36.7: canonical device, protocol, interrupts, DMA, device interaction, drivers)	405–412	The device model behind every Kagan driver.
SECONDARY	MOS	§5.1 *Principles of I/O Hardware*, especially §5.1.3 *Memory-Mapped I/O*	337–351	Why x86 has both port and memory-mapped I/O.
SECONDARY	MOS	§5.6.2 *Output Software* (text windows, escape sequences)	399–405	How a text console turns bytes into a screen.

Weight	Source	Section	Pages	Relevance
SECONDARY	OSDI	§3.8.6 *Implementation of the Display Driver*	357–366	MINIX writing to video memory and scrolling.
OPTIONAL	OSC	§12.2 *I/O Hardware*	490–500	Another view: polling, interrupts, DMA.
OPTIONAL	MOS	§1.8 *The World According to C* (if skipped in Phase 0)	73–77	Organising headers for a multi-file project.
PRIMARY	LKP	Ch. 4, *Understanding kernel logging and printk* (ring buffer, log levels, `pr_*` macros, `pr_fmt`)	173–194	The logging design of Task 2.7.

Prior art: MINIX 3

Open `drivers/tty/console.c (15900)` in OSDI. MINIX keeps a per-console struct with the cursor position and the video memory address, and it scrolls either by copying memory or by moving the hardware's display start. Its `out_char` and `scroll_screen` correspond to Task 2.5. The MINIX console driver runs as a separate process and receives characters in messages; in Kagan it is an ordinary function call.

Linux, for comparison. LKP Linux's `printk` does not write to the screen directly. It appends every message, tagged with a log level, to a ring buffer in memory; console drivers drain the buffer, and `dmesg` reads it back long after the screen has scrolled. Task 2.7 copies that design.

4 Implementation

Task 2.1 Give the loader a stack and call C

Reserve 16 KiB in `.bss`, point ESP at its top (the stack grows down), push the two Multiboot values in reverse order and call `kmain`. Also set bit 1 of the Multiboot flags so GRUB fills in the memory size fields, and bit 0 so boot modules are page-aligned (modules are used in Phase 10).

boot/loader.s

```
global loader
extern kmain

MB_ALIGN     equ 1 << 0                ; align boot modules on 4 KiB pages
MB_MEMINFO   equ 1 << 1                ; ask for mem_lower / mem_upper and the memory map
MAGIC_NUMBER equ 0x1BADB002
FLAGS        equ MB_ALIGN | MB_MEMINFO
CHECKSUM     equ -(MAGIC_NUMBER + FLAGS)

KERNEL_STACK_SIZE equ 16384

section .multiboot
align 4
    dd MAGIC_NUMBER
    dd FLAGS
    dd CHECKSUM

section .bss
align 16
kernel_stack_bottom:
    resb KERNEL_STACK_SIZE
kernel_stack_top:

section .text
```

```
loader:
    mov  esp, kernel_stack_top ; C needs a stack before anything else
    push ebx                   ; arg 2: physical address of the Multiboot info
    push eax                   ; arg 1: magic value 0x2BADB002
    call kmain                 ; kmain(magic, mbi)
.hang:
    cli                        ; kmain should never return; if it does, stop
    hlt
    jmp  .hang
```

Task 2.2 The first C code

Start with the smallest C that proves the jump worked: write a character straight into video memory.

kernel/kmain.c (first version)

```
#include <stdint.h>

void kmain(uint32_t magic, uint32_t mbi_addr)
{
    (void)magic; (void)mbi_addr;
    volatile uint16_t *vga = (volatile uint16_t *)0xB8000;
    vga[0] = 0x1F00 | 'T';      /* white on blue, top-left corner */
    vga[1] = 0x1F00 | 'K';
}
```

Verify: A blue “TK” appears in the top-left corner. In GDB, `break kmain` stops there and `p/x magic` prints `0x2badb002`.

Task 2.3 Compile C for the kernel

Extend the Makefile with a C rule. The flags say: freestanding (no hosted library assumptions), warnings on and treated as errors, debug info for GDB, and keep frame pointers for backtraces in Phase 3. Link through `gcc` so that `libgcc` is available; GCC sometimes emits calls into it (64-bit division, for example).

The extra warnings each close one class of bug while the kernel is still small enough to fix every warning cheaply. `-Wformat=2` checks every format string against its arguments, `-Wshadow` catches a local that hides a global of the same name, `-Wvla` forbids variable-length arrays (a length from outside can push one past the end of a 16 KiB kernel stack), and `-Wstrict-prototypes` rejects `void f()`, which in C means “any arguments” rather than “none”. `-Werror` makes them stick: a warning that may be ignored will be ignored.

Makefile (additions)

```
CC      := i686-elf-gcc
CFLAGS  := -std=gnu11 -ffreestanding -O2 -g -Wall -Wextra -fno-omit-frame-pointer -Iinclude \
           -Werror -Wformat=2 -Wshadow -Wvla -Wstrict-prototypes
LDFLAGS := -T link.ld -ffreestanding -nostdlib
C_SRCS  := $(wildcard kernel/*.c kernel/drivers/*.c lib/*.c)
OBJS    := $(BUILD)/boot/loader.o $(patsubst %.c,$(BUILD)/%.o,$(C_SRCS))

$(BUILD)/kernel.elf: $(OBJS) link.ld
    $(CC) $(LDFLAGS) -o $@ $(OBJS) -lgcc

$(BUILD)/%.o: %.c
    @mkdir -p $(dir $@)
    $(CC) $(CFLAGS) -c $< -o $@

run: $(BUILD)/kernel.elf
    qemu-system-i386 -kernel $< -serial stdio
```

Task 2.4 Port I/O helpers

C has no way to express `in` and `out`, so wrap them. LOB writes them as separate assembly functions, which exercises cdecl directly (the port is at `[esp+4]`, the value at `[esp+8]`). Most kernels use GCC inline assembly instead, which avoids a function call per port access. Write the assembly version first as a cdecl check, then switch to this header:

include/io.h

```c
#pragma once
#include <stdint.h>

static inline void outb(uint16_t port, uint8_t val)
{
    __asm__ volatile ("outb %0, %1" : : "a"(val), "Nd"(port));
}

static inline uint8_t inb(uint16_t port)
{
    uint8_t ret;
    __asm__ volatile ("inb %1, %0" : "=a"(ret) : "Nd"(port));
    return ret;
}

/* A write to an unused port takes about 1 microsecond: a tiny delay for old devices. */
static inline void io_wait(void) { outb(0x80, 0); }
```

Task 2.5 The VGA text driver

Write `kernel/drivers/vga.c` with `vga_init`, `vga_clear`, `vga_set_color`, `vga_putc` and `vga_write`. `vga_putc` must handle `\n` (new line), `\r`, `\t` (next multiple of 8), and `\b` (backspace, needed in Phase 4). When the cursor passes the last row, scroll: move rows 1–24 up by one with `memmove` and clear the last row. After each character, move the hardware cursor.

kernel/drivers/vga.c (core pieces)

```c
#define VGA_WIDTH  80
#define VGA_HEIGHT 25
static volatile uint16_t *const vga_buf = (volatile uint16_t *)0xB8000;
static uint8_t  color = 0x07;             /* light grey on black */
static uint16_t row, col;

static inline uint16_t vga_cell(char c, uint8_t attr)
{
    return (uint16_t)attr << 8 | (uint8_t)c;
}

static void move_cursor(void)
{
    uint16_t pos = row * VGA_WIDTH + col;
    outb(0x3D4, 14); outb(0x3D5, pos >> 8);    /* cursor location high byte */
    outb(0x3D4, 15); outb(0x3D5, pos & 0xFF);  /* cursor location low byte  */
}
```

Verify: Print 100 numbered lines: the screen scrolls and the last 25 are visible. The cursor blinks after the last character.

Task 2.6 The serial driver

Write `kernel/drivers/serial.c`. The initialisation sequence below sets 38 400 baud 8N1 with FIFOs enabled, following LOB chapter 4. Each register offset is relative to the base port `0x3F8`.

kernel/drivers/serial.c

```c
#define COM1 0x3F8

void serial_init(void)
{
    outb(COM1 + 1, 0x00);   /* interrupt enable register: all off (we poll)        */
    outb(COM1 + 3, 0x80);   /* line control: set DLAB to access the divisor         */
    outb(COM1 + 0, 0x03);   /* divisor low byte:  115200 / 3 = 38400 baud           */
    outb(COM1 + 1, 0x00);   /* divisor high byte                                     */
    outb(COM1 + 3, 0x03);   /* line control: 8 data bits, no parity, 1 stop, DLAB=0 */
    outb(COM1 + 2, 0xC7);   /* FIFO control: enable, clear, 14-byte threshold       */
    outb(COM1 + 4, 0x03);   /* modem control: DTR and RTS set                        */
}

static int tx_empty(void) { return inb(COM1 + 5) & 0x20; }  /* line status bit 5 */

void serial_putc(char c)
{
    if (c == '\n')
        serial_putc('\r');  /* terminals expect CR LF */
    while (!tx_empty())
        ;
    outb(COM1, (uint8_t)c);
}
```

Verify: With `make run` (which passes `-serial stdio`), text sent to the serial port appears in the host terminal. Also try `-serial file:build/serial.log`.

Portability note

This UART comes back in Phase 17. On QEMU's RISC-V `virt` board it is the same 16550, with the same registers at the same offsets, but memory-mapped at `0x10000000` instead of reached through ports. So keep every register access in two small functions, say `uart_read(reg)` and `uart_write(reg, val)` on top of `inb` and `outb`, and write the rest of `serial.c` with them. Phase 16 then swaps only those two for MMIO accessors, and both machines share the rest of the driver.

Task 2.7 kprintf and the console layer

Write `lib/printf.c` with `kvprintf(void (*putc)(char), const char *fmt, va_list ap)` as the engine, and `kprintf` on top of it using the console. Support `%c`, `%s`, `%d`, `%i`, `%u`, `%x`, `%p` and `%%`, plus zero-padding with a width (`%08x`), which makes addresses readable. Use `utoa` and `itoa` from Phase 0.

Declare `kprintf` with GCC's `format` attribute. GCC then checks every call the way it checks `printf`: `kprintf("%s", 42)` stops the build instead of printing memory at address 42. One consequence is easy to miss. On `i686-elf`, `uint32_t` is `unsigned long`, not `unsigned int`, so GCC wants `%lx` and `%lu` for it. Accept the `l` modifier in `kvprintf` and treat it like no modifier, since both types are 32 bits here.

lib/printf.c (skeleton)

```c
#include <stdarg.h>

/* in include/kprintf.h: argument 1 is the format, checking starts at argument 2 */
void kprintf(const char *fmt, ...) __attribute__((format(printf, 1, 2)));

void kprintf(const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
```

```
    kvprintf(console_putc, fmt, ap);
    va_end(ap);
}
```

Then give every message a **log level** and keep a copy of everything in memory, the way Linux's `printk` does (LKP ch. 4). A message goes first into a ring buffer, `log_buf`, and only then to the console, and only if its level is at least as urgent as `console_loglevel`. When the screen has scrolled away or no serial cable is attached, the buffer still holds the whole boot; in Phase 4 a `dmesg` monitor command prints it. Each source file can define `pr_fmt` to prefix its messages with the subsystem's name.

include/log.h

```
enum { LOG_ERR = 3, LOG_WARN = 4, LOG_INFO = 6, LOG_DEBUG = 7 };    /* Linux's numbers */
extern int console_loglevel;            /* messages with level <= this reach the console */

void klog(int level, const char *fmt, ...)    /* always into log_buf, maybe to the console */
    __attribute__((format(printf, 2, 3)));

#ifndef pr_fmt
#define pr_fmt(fmt) fmt                 /* e.g. #define pr_fmt(fmt) "pmm: " fmt */
#endif
#define pr_err(fmt, ...)   klog(LOG_ERR,   pr_fmt(fmt), ##__VA_ARGS__)
#define pr_warn(fmt, ...)  klog(LOG_WARN,  pr_fmt(fmt), ##__VA_ARGS__)
#define pr_info(fmt, ...)  klog(LOG_INFO,  pr_fmt(fmt), ##__VA_ARGS__)
#define pr_debug(fmt, ...) klog(LOG_DEBUG, pr_fmt(fmt), ##__VA_ARGS__)
```

Make `log_buf` a 16 KiB array of characters with a write index that wraps; storing the level and the uptime in front of each line is a natural extension once the timer exists. Keep plain `kprintf` for the crash paths: `panic` must print even if the log code is broken.

Verify: Write a test that prints `0`, `-1`, `INT_MIN`, `0xFFFFFFFF` with `%u` and `%x`, `%08x` of `0xB8000`, a `NULL` string (print `(null)`), and `%%`. Compare with what the same format string gives in a Linux test program. With `console_loglevel` set to `LOG_INFO`, a `pr_debug` line stays off the screen but is present in `log_buf`, and writing more than 16 KiB of messages keeps the newest ones. Finally, add `kprintf("%s\n", 42);` to `kmain`: `make` must fail with a `-Wformat` error. Remove the line again.

Task 2.8 panic and ASSERT

When the kernel detects something impossible, it must stop loudly instead of continuing with corrupt state. `panic` prints the message with file and line, disables interrupts and halts. `ASSERT` calls it when a condition is false. Use `ASSERT` generously throughout the kernel; it is the cheapest debugging tool available.

include/panic.h

```
#pragma once
void panic_at(const char *file, int line, const char *fmt, ...)
    __attribute__((noreturn, format(printf, 3, 4)));

#define panic(...)  panic_at(__FILE__, __LINE__, __VA_ARGS__)
#define ASSERT(cond) \
    do { if (!(cond)) panic("assertion failed: %s", #cond); } while (0)
```

`panic` is for broken invariants: states the kernel's own code should never reach. It is never the answer to bad input. A wrong argument from a caller, a full table or a failed allocation is an ordinary error, returned as a negative `errno` value such as `-EINVAL` or `-ENOMEM`. From Phase 10 on, a user program decides what reaches the kernel, and a kernel that panics on bad input lets any program switch it off. Record that policy, together with a short list of coding rules taken from the SEI CERT C standard, as a second section of `docs/security/threat-model.md`. Each rule names its CERT identifier for reference.

docs/security/threat-model.md (second section)

```
## Coding rules

Errors: return a negative errno (-EINVAL, -ENOMEM); 0 or a positive
value means success. Never ignore a return value: allocators carry
__attribute__((warn_unused_result)). panic() only for broken
invariants, never because of input from outside.     (ERR33-C)

INT30-C, INT32-C  sizes and offsets from outside go through
                  lib/checked.h; no -fwrapv as a substitute
ARR30-C           no index without a bound check
STR31-C, STR32-C  strings from outside are bounded and always
                  NUL-terminated
MEM30-C, MEM31-C  every allocation is checked; a freed pointer
                  is never used again
EXP33-C           no reads of uninitialised memory; zero a struct
                  before it leaves the kernel
INT31-C, INT34-C  explicit widths (uint32_t); shift counts stay
                  below the width of the type
FIO30-C           format strings are literals; -Wformat=2 -Werror
CON               shared data has a lock; no lock held across sleep
```

Task 2.9 Read the Multiboot information

Check that `magic` equals `0x2BADB002`; otherwise panic with “not loaded by a Multiboot loader”. Define the beginning of the Multiboot information structure as a packed struct (see LOB “Packing Structs” and the Multiboot spec §3.3). If bit 0 of `flags` is set, `mem_lower` and `mem_upper` hold the KiB of memory below 1 MiB and above 1 MiB. Print a banner: `Kagan 0.2 - 127 MiB RAM`.

include/multiboot.h (first fields)

```
struct multiboot_info {
    uint32_t flags;
    uint32_t mem_lower;       /* KiB below 1 MiB, valid if flags bit 0 */
    uint32_t mem_upper;       /* KiB above 1 MiB, valid if flags bit 0 */
    uint32_t boot_device;
    uint32_t cmdline;         /* physical address of the command line  */
    uint32_t mods_count;      /* boot modules (Phase 10)                */
    uint32_t mods_addr;
    uint32_t syms[4];
    uint32_t mmap_length;     /* memory map (Phase 5)                   */
    uint32_t mmap_addr;
} __attribute__((packed));
```

Verify: `qemu-system-i386 -kernel build/kernel.elf -serial stdio -m 128` reports roughly 127 MiB. Changing `-m` changes the reported number accordingly.

5 Testing and debugging

From this phase on, run QEMU with `-serial stdio` every time and put a `kprintf` at each boot step (“GDT loaded”, “paging on”, …). When the kernel dies, the last line in the log shows where.

Symptom	Likely cause	Resolution
QEMU reboots in a loop right after `call kmain`	Stack not set, or ESP pointing at the wrong end of the reserved area	Run with `-d int,cpu_reset -no-reboot` to see the fault; check `esp` in GDB before the `call`.
Linker: `undefined reference to memset` (or `memcpy`)	GCC emits these calls itself for struct copies and array initialisation, even when freestanding	Provide them in `lib/string.c`. They must exist in every kernel.

Symptom	Likely cause	Resolution
Linker: `undefined reference to __stack_chk_fail`	Host compiler with the stack protector enabled	Use the cross compiler (or add `-fno-stack-protector`).
Nothing appears on screen, no crash	Missing `volatile`, so the optimiser removed the stores	Declare the buffer pointer as `volatile uint16_t *`.
Serial output is garbled or missing	Wrong divisor or line settings, or not waiting for the transmitter	Re-check the register sequence; poll bit 5 of the line status register.
`%d` prints wrong values for negative numbers	Converting `INT_MIN` by negating an `int` overflows	Negate into an `unsigned` before converting.
Build fails: `%x expects argument of type unsigned int` on a `uint32_t`	On `i686-elf`, `uint32_t` is `unsigned long`	Use `%lx` and `%lu`, which `kvprintf` accepts (Task 2.7).
Build fails: `function declaration isn't a prototype`	`-Wstrict-prototypes` and a declaration like `void vga_clear()`	Write `void vga_clear(void)`.

6 Exit criteria

Exit criteria: The kernel can talk

- ☐ `kmain` runs on the kernel's own 16 KiB stack and receives the Multiboot magic and information pointer.
- ☐ The screen shows a coloured Kagan banner with the RAM size, and scrolling works.
- ☐ The same messages appear in the host terminal through the serial port.
- ☐ `kprintf` passes the edge-case tests; `panic` and `ASSERT` print file and line and halt.
- ☐ The Makefile builds C and assembly, links with `libgcc`, and `make run` opens serial on stdio.
- ☐ The kernel builds without a single warning under `-Werror`, and a wrong format string stops the build.
- ☐ The coding rules and the error policy are committed in `docs/security/threat-model.md`.

Review questions

1. Why must ESP be set before `call kmain`, and why does it point at the *end* of the reserved area?
2. Which does the VGA text buffer use, memory-mapped or port-mapped I/O? Which does the cursor use?
3. Why is `stdarg.h` available in a freestanding kernel when `stdio.h` is not?
4. What does `__attribute__((packed))` do, and what would go wrong without it in `multiboot_info`?
5. Why might GCC call `memcpy` in a kernel that never calls it explicitly?
6. What is the serial divisor for 115 200 baud? For 9 600?

7 Extensions

Add log levels (`INFO`, `WARN`, `ERROR`) with different colours on screen and a prefix on serial. Keep a copy of every message in a ring buffer in memory; Phase 4 adds a `dmesg` command that prints it. Turkish letters need separate handling: code page 437 contains ç, ö, ü, Ç, Ö and Ü but not ğ, ş, ı, İ, Ğ or Ş. Write a small UTF-8 to CP437 converter for the letters that exist and substitute the rest for now. Phase 15 offers two ways to get full Turkish text: a custom VGA font, or a graphical framebuffer with a bundled font.

Next: **Phase 3 Segmentation, the IDT and CPU Exceptions** →

References

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at `https://littleosbook.github.io`.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at `https://pages.cs.wisc.edu/~remzi/OSTEP/`.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

LOB and **OSTEP** are freely available; the roadmap lists ISBNs for the others. Where sources disagree, the design follows the newer one when its method also suits Kagan. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free **OSTEP** chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in **OSDI** Appendix B.